MATH DIRECTED READING PROGRAM / MEETING 1

From Familiar Linear Algebra to Neural Networks

Self-contained guided reading and problem sheet | Revision 3.0

Aim. Learn to read the language of a small neural network as ordinary mathematics: functions, coordinates, compositions, and choices of coefficients. After each calculation, explain what it says about the model.

Background. Matrix multiplication, affine functions, rank as the dimension of an image, and elementary piecewise-defined functions. No previous machine-learning course, programming, or probability is needed. Textbook readings at the end are optional support, not missing prerequisites.

Pacing. Read the local setup before attempting its questions. This is one learning module, not a requirement to finish every page in one hour. A natural stopping point is after Problem 1.2. Use a separate notebook for your calculations.

Using the self-checks. After each numbered problem, try its short self-checks with the worked solution covered. Definitions may stay open. Give a reason, not only a yes/no answer; these checks guide discussion rather than add a timed test.

1.1 A network made only of affine maps

REVIEW: A FAMILIAR MAP, WITH NEW NAMES

An **affine map** has the form $x \mapsto Wx + b$. The matrix W gives the linear part and the vector b gives the translation. It is a linear map when $b = 0$.

This computation is an **affine layer**. In common ML usage it is also called a **linear layer**, even when the bias is nonzero; the mathematical map is then affine, not strictly linear. The entries of W are **weights**, and b is a **bias vector**. We call their adjustable scalar entries **parameters**. A **model** is a function used to produce a prediction. Writing f_θ records its dependence on a collection of parameters θ; varying θ gives a family of functions.

We use **column vectors throughout**. Let d, H, m be positive integers. Consider

$$x \in \mathbb{R}^d \longmapsto u = W_1 x + b_1 \in \mathbb{R}^H \longmapsto \widehat{y} = W_2 u + b_2 \in \mathbb{R}^m,$$

where

$$W_1 \in \mathbb{R}^{H \times d}, \quad b_1 \in \mathbb{R}^H, \qquad W_2 \in \mathbb{R}^{m \times H}, \quad b_2 \in \mathbb{R}^m.$$

Here $\theta = (W_1, b_1, W_2, b_2)$. The vector x is the **input**, and $\widehat{y}$ is the **output** or **prediction**. The intermediate vector u is a **hidden representation**: it is computed inside the model rather than supplied as an input or returned as the final output. The collection of its H coordinate functions is the **hidden layer**; these coordinates are the **hidden units**. Its **width** is the integer H.

Thus, "hidden width H" means that this intermediate vector has H coordinates. It does *not* assert that W_1 has rank H. In this sheet, **depth** counts parameterized affine stages, so this example has two such stages and one hidden layer.

1.1 / COMPUTE, PROVE, AND INTERPRET

WORKED EXAMPLE: TWO STAGES, ONE INTERMEDIATE NUMBER

Take $d = m = 2$, $H = 1$, and

$$W_1 = \begin{pmatrix} 1 & 2 \end{pmatrix}, \quad b_1 = 1, \qquad W_2 = \begin{pmatrix} 3 \\ -1 \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

For $x = (1, 2)^\top$, the intermediate number is $u = 1 + 2 \cdot 2 + 1 = 6$, and $\widehat{y} = (18, -4)^\top$. Evaluating these stages in order, with the parameters held fixed, is called a **forward pass**. Here two input coordinates are replaced by one intermediate number before two output coordinates are computed.

Guided questions

(a) **Decode the description.** For the worked example, identify the input dimension, hidden width, output dimension, and all parameters. Then evaluate the model at $x = (0, 0)^\top$ and at $x' = (2, -1)^\top$. Do the two different inputs produce different hidden representations?

(b) **Compose the maps.** For general W_1, b_1, W_2, b_2, expand

$$f_\theta(x) = W_2(W_1 x + b_1) + b_2$$

into $Cx + c$. Identify C, c and check their dimensions. What familiar class of functions still contains the entire model?

(c) **Add another stage.** Compose $Cx + c$ with one more affine map $v \mapsto Dv + e$. What changes in the formula, and what does not? Explain why any finite stack made only of affine stages is still affine. A formal induction is optional.

RANK REMINDER BEFORE THE WIDTH QUESTION

For a matrix M, $\operatorname{rank} M = \dim \operatorname{im} M$, where $\operatorname{im} M = \{Mv : v \text{ is an input vector}\}$. If S is a vector subspace and B is linear, then $\dim B(S) \leq \dim S$. A subspace of $\mathbb{R}^H$ has dimension at most H.
For an affine map $Cx + c$, the rank statement below concerns its *linear part* C. Translation by c does not change the dimension of its affine image.

(d) **Derive the width bound.** Use $\operatorname{im}(W_2W_1) = W_2(\operatorname{im} W_1)$ and $\operatorname{im}(W_2W_1) \subseteq \operatorname{im} W_2$ to show

$$\operatorname{rank}(W_2W_1) \leq \min\{\operatorname{rank} W_1, \operatorname{rank} W_2\} \leq H.$$

Is it necessary to assume that the two matrices have equal ranks? Does the bound require $H < d$ and $H < m$, or do those inequalities merely make it a stricter restriction?

(e) **Give the bound a concrete meaning.** Could a model with dimensions $3 \to 2 \to 3$, made only of affine stages, equal $x \mapsto x$ on all of $\mathbb{R}^3$? Explain. More generally, if $W_1x = W_1x'$, why can no later computation that receives only u distinguish x from x'?

Translate back into machine-learning language. Complete both statements with a mathematical justification: "Adding more affine stages cannot produce a non-affine input–output map." "A narrow intermediate representation can restrict which linear parts the model can represent." These are statements about functions, not about how an algorithm finds their parameters.

SELF-CHECK 1.1

1. A model has dimensions $4 \to 2 \to 3$ and contains only affine stages. Can its overall linear part have rank 3? Can it have rank 1? Explain why width gives a bound rather than an equality.
2. Keep the architecture and all its weights fixed, but replace one input by another. Which quantities may change: hidden width, hidden representation, bias entries, or prediction? Name their different roles.

1.2 Insert a nonlinear activation

SETUP: A NEW OPERATION BETWEEN THE TWO MATRICES

Define the **rectified linear unit**, abbreviated **ReLU**, by

$$\mathrm{ReLU}(t) = \max\{0, t\} = \begin{cases} 0, & t \le 0, \\ t, & t > 0. \end{cases}$$

Here we use an **elementwise activation**: a scalar function applied separately to each coordinate of an intermediate vector. ReLU is one such activation. Other activations, such as the softmax introduced in Meeting 3, act on several coordinates jointly. For example, $\mathrm{ReLU}((-2, 0, 3)^\top) = (0, 0, 3)^\top$. Here the function ReLU is fixed; we do not train or choose a new scalar function for each input.

For general dimensions d, H, m as in Problem 1.1, now compute

$$u = W_1 x + b_1, \qquad h = \mathrm{ReLU}(u), \qquad \widehat{y} = f_\theta(x) = W_2 h + b_2.$$

The entries of $u \in \mathbb{R}^H$ are **pre-activations**, meaning values before applying ReLU. The entries of $h \in \mathbb{R}^H$ are **hidden activations**, meaning the resulting values. Here h is the hidden representation, and its width is again H.

If $w_j^\top$ is row j of W_1, the jth hidden unit computes

$$h_j(x) = \mathrm{ReLU}\big(w_j^\top x + (b_1)_j\big).$$

This scalar computation is also called a **neuron**. The model has one hidden layer, containing H such units. The output stage in this example is affine, with *no* ReLU after it.

The arrangement of stages, their dimensions, and the choice of activation are the **architecture**. The tuple $\theta = (W_1, b_1, W_2, b_2)$ contains the numerical parameters. For now, the architecture is fixed. **Trainable** means permitted to be adjusted when fitting examples; it does not mean already fitted.

Guided questions

(a) **Read coordinates as units.** Write a formula for $\widehat{y}_i$ as a sum over $j = 1, \dots, H$. Identify where a hidden unit is computed and where the output combines hidden units. Check the dimensions of $u, h, \widehat{y}$.

(b) **Carry out a forward pass.** Use

$$W_1 = \begin{pmatrix} 1 & -1 \\ 2 & 1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 1 & -2 \end{pmatrix}, \quad b_2 = 0.$$

Compute $u, h, \widehat{y}$ for $x = (2, 1)^\top$, and then for $x = (0, 0)^\top$. In the second calculation, compare the output with the value obtained by omitting ReLU.

(c) **Count adjustable numbers.** A $p \times q$ matrix has pq entries. Assuming all four arrays are unrestricted and trainable, count the scalar parameters for general d, H, m, and for the numerical example. Count entries allowed to vary, even when their current value is zero. Does increasing the number of input examples change this count when the architecture stays fixed?

(d) **Locate the change in the mathematics.** If we replace $\mathrm{ReLU}(t)$ by t, which conclusion of Problem 1.1 returns? Using $d = H = m = 1$, give parameter choices producing a non-affine function and an affine function. Does the presence of ReLU force every parameter choice to produce a non-affine function?

Translate back into machine-learning language. Describe a forward pass using "weights," "bias," "hidden activation," and "output." Separate values that change when x changes from parameters that are being held fixed.

> **SELF-CHECK 1.2**
>
> 1. For a scalar unit with pre-activation $u = 2x - 1$, compare the pre-activation and activation at $x = 0$ and $x = 1$. Which numbers are computed values, and which are fixed parameters?
> 2. A network contains ReLU, but its output matrix is zero. Must its output still be non-affine? Explain what the architecture permits versus what these particular parameters produce.

1.3 Build a function by combining hidden units

REVIEW: SHIFTED RELU FUNCTIONS ARE HINGES

For a real number τ, the function $t \mapsto \mathrm{ReLU}(t - \tau)$ is zero to the left of τ and equals $t - \tau$ to its right. Its slope changes from 0 to 1 at τ. This graph is called a **hinge**; the point τ is a **breakpoint**. Multiplying by a scalar a changes the right-hand slope to a.

A **continuous piecewise-affine function** is continuous and agrees with an affine formula on each of several intervals; its slope may change at their endpoints. We use slopes only inside those intervals, not derivatives at the breakpoints.

Small example. The function $\mathrm{ReLU}(t) - \mathrm{ReLU}(t - 2)$ equals 0 for $t \le 0$, equals t for $0 < t < 2$, and equals 2 for $t \ge 2$. A weighted sum of two hidden-unit outputs has produced a function with two changes in slope.

Our target is the **tent function**

$$T(t) = \begin{cases} 0, & t \le -1, \\ t + 1, & -1 < t \le 0, \\ 1 - t, & 0 < t < 1, \\ 0, & t \ge 1. \end{cases}$$

We will construct it by using three fixed hidden units and choosing their output weights:

$$h_1(t) = \mathrm{ReLU}(t + 1), \quad h_2(t) = \mathrm{ReLU}(t), \quad h_3(t) = \mathrm{ReLU}(t - 1), \qquad F(t) = \alpha h_1(t) + \beta h_2(t) + \gamma h_3(t).$$

Guided questions

(a) **Inspect the building blocks.** Sketch h_1, h_2, h_3. For each of the four intervals separated by $-1, 0, 1$, record which functions are zero and which have slope 1.

(b) **Choose output weights.** Express the slope of F on each interval in terms of α, β, γ. Match these slopes to those of T, in order from left to right, and solve for the three weights. Why do matching slopes, continuity, and agreement on the leftmost interval establish $F = T$ everywhere?

(c) **Check a few inputs.** Evaluate your formula at $-2, -\frac{1}{2}, 0, \frac{1}{2}, 2$. Explain why no single affine function can equal T on the whole real line.

(d) **Translate into network notation.** Write

$$T(t) = W_2\,\mathrm{ReLU}(W_1 t + b_1) + b_2$$

with explicit matrices and biases. Give d, H, m, and identify each hidden unit. Which parameters did we keep fixed while choosing the output weights in part (b)?

Translate back into machine-learning language. Explain this construction once as "a linear combination of three piecewise-affine functions" and once as "a one-hidden-layer ReLU network of width three." What mathematical object do the two descriptions refer to? Constructing weights by hand is not yet an automatic training procedure.

> **SELF-CHECK 1.3**
>
> 1. Using your network for $T(t)$, change only its output stage to represent $2T(t) - 1$. Give the new output weights and bias. Has the hidden width changed?
> 2. In the tent construction, which parameter choices locate the breakpoints and which combine their slope changes? Explain without introducing any new kind of mathematical object.

1.4 From evaluating a model to fitting examples

SETUP: WHAT IS A LEARNING PROBLEM?

A **training example** is a pair (x_s, y_s): an input and its observed or desired output, called the **target**. A **training dataset** is a collection $\mathcal{D} = \{(x_s, y_s) : s = 1, \ldots, N\}$. The symbol N counts examples; it is not a layer width. In this problem, inputs and targets are scalars.

For a model prediction $\widehat{y} = f_\theta(x)$ and target y, a **loss** assigns a numerical penalty to their disagreement. We choose the squared-error loss

$$\ell(\widehat{y}, y) = \tfrac{1}{2}(\widehat{y} - y)^2, \qquad L_{\mathcal{D}}(\theta) = \frac{1}{N}\sum_{s=1}^{N} \ell(f_\theta(x_s), y_s).$$

The latter is the **average training loss**. It is a function of the parameters with the examples held fixed. **Training**, or **fitting**, means using these examples to choose parameters with small training loss. A training algorithm is a procedure for making that choice; such procedures are a later topic.

One numerical check. If the target is $y = 1$ and the prediction is $\widehat{y} = \frac{1}{2}$, then the loss on that example is $\frac{1}{2}(\frac{1}{2} - 1)^2 = \frac{1}{8}$. This is a numerical comparison, not a probability.

Setup for the questions. Use the three examples

$$\mathcal{D} = \{(-1, 0), (0, 1), (1, 0)\}.$$

A function **interpolates** these data when it agrees with all three targets exactly. We compare two families: affine functions $x \mapsto ax + b$, and the width-three ReLU networks from Problem 1.3. For the latter family, all four parameter arrays may vary, although a particular construction may hold some entries fixed.

Guided questions

(a) **Separate the roles.** Identify the data, parameters, prediction, and loss. Write $L_{\mathcal{D}}(a, b)$ explicitly for the affine family. In a forward pass, what is held fixed? When fitting parameters, what is held fixed instead?

(b) **Understand zero training loss.** Show that $L_{\mathcal{D}} = 0$ exactly when the model interpolates all three examples. Why is nonnegativity of the three summands important?

(c) **Compare the two families.** Show that no affine function interpolates $\mathcal{D}$, while the tent network does. Which observation is a limitation of a function family, rather than a failure of a training algorithm?

(d) **Test what the data determine.** Consider the family

$$S_c(x) = \mathrm{ReLU}(x+1) - 2\,\mathrm{ReLU}(x) + c\ \mathrm{ReLU}(x-1), \qquad c \in \mathbb{R}.$$

Evaluate S_c at all three training inputs and at $x = 2$. Which values of c give zero training loss? Do they all give the same prediction at 2? Verify that these are networks with the *same* width and hidden units; only one output weight changes.

Translate back into machine-learning language. Generalization concerns a model's predictions on new examples from the task, not just those used for fitting. Explain why perfect agreement on this finite dataset does not, by itself, select a unique prediction at 2. Without a target at 2 or an additional assumption, we have not shown which such prediction is better.

SELF-CHECK 1.4

1. For the family S_c, suppose one extra training example is $(2, 3)$. Which value of c now fits all four examples? What new information did that target supply?
2. A model has zero loss on the original three examples. Does that show that a training algorithm found those parameters, or that its prediction at every new input is correct? Separate the two claims.

Discussion checkpoint

READY TO MOVE ON WHEN...

Given a small network, identify weights, biases, pre-activations, hidden activations, width, and output; carry out a forward pass; explain why a stack of affine maps remains affine and why matching finite training data does not determine every new prediction. Use 1.2 and 1.4 as representative tasks.
You may consult definitions and notation. Explain the choices without following a worked solution line by line. If a self-check exposes a gap, return to that problem's setup; optional extensions do not block progress.

Bring one numerical computation and one short argument that you can explain without relying on undefined terminology. A possible five-minute explanation begins with

$$\text{a composition of affine maps} \quad \longrightarrow \quad \text{a ReLU network} \quad \longrightarrow \quad \text{a family fitted to data.}$$

Explain where a new *operation* was introduced, where familiar mathematics was merely given a new *name*, and where a new *problem*—choosing parameters from data—was posed.

CHECK YOUR TRANSLATIONS

"Hidden width three" counts intermediate coordinates, not matrix rank.
"Forward pass" evaluates a fixed model; "training" chooses its parameters from examples.
"More affine stages" still yields an affine map. A nonlinear activation permits—but does not force—a non-affine map.
"Zero training loss" describes the specified finite data, not all inputs.

Selected hints

Try the relevant question before reading its hint.

1.1(d). For the first inequality, apply the dimension reminder to $W_2(\operatorname{im} W_1)$, and separately use containment in $\operatorname{im} W_2$. The number of rows or columns limits rank but need not equal it.

1.1(e). The identity on $\mathbb{R}^3$ has a linear part of rank three. For the second question, identical intermediate vectors supply identical inputs to every later function.

1.2(c). Count the entries of W_1, b_1, W_2, b_2 separately. The dimensions d, H, m are fixed choices here, not additional trainable scalar entries.

1.2(d). For a non-affine example, take one scalar hidden unit computing $\mathrm{ReLU}(x)$. For an affine example, consider what happens if the output weights are all zero.

1.3(b). On the intervals $(-1, 0)$, $(0, 1)$, and $(1, \infty)$, the slopes are respectively α, $\alpha + \beta$, and $\alpha + \beta + \gamma$.

1.4(d). The last hidden unit is zero at each training input, but not at 2. Keep track of its output weight rather than changing the architecture.

Optional reading: reconnect with the textbook

[1] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Author's book and teaching-materials site. The following locators use the 2023 first-edition organization; use the section titles if another version renumbers them.

For Problems 1.1–1.2, consult **§3.5, Terminology**, and **§§4.3–4.4, Deep neural networks / Matrix notation**. For the construction in Problem 1.3, consult **§3.1, Neural network example**, and **§3.4, Shallow neural networks: general case**. For Problem 1.4, consult **§§2.1–2.2, Supervised learning overview / Linear regression example**. You do not need the universal approximation theorem for this sheet. For the broader use of the term activation, compare the PyTorch `Softmax` definition: normalization acts over a specified set of coordinates.

These are original guided exercises and examples for the DRP, not reproduced textbook exercises. The local definitions fix this sheet's notation and layer-counting convention. Revision 3.0 adds problem-level self-checks and a meeting-level completion criterion. The companion mentor notes contain response criteria, not required student reading.